Digital Banking Fraud Detection & Simulation Engine

Technical Architecture & Module Specification

---

1 Project Overview

1.1 Project Title

Digital Banking Fraud Detection & Simulation Engine

1.2 Objective

To build a modular online banking fraud detection system that:

- Simulates real-world banking transactions
- Detects fraudulent behavior using rule-based logic
- Integrates a Machine Learning fraud prediction model
- Stores alerts and risk scores
- Displays real-time fraud monitoring dashboards

1.3 Final Output

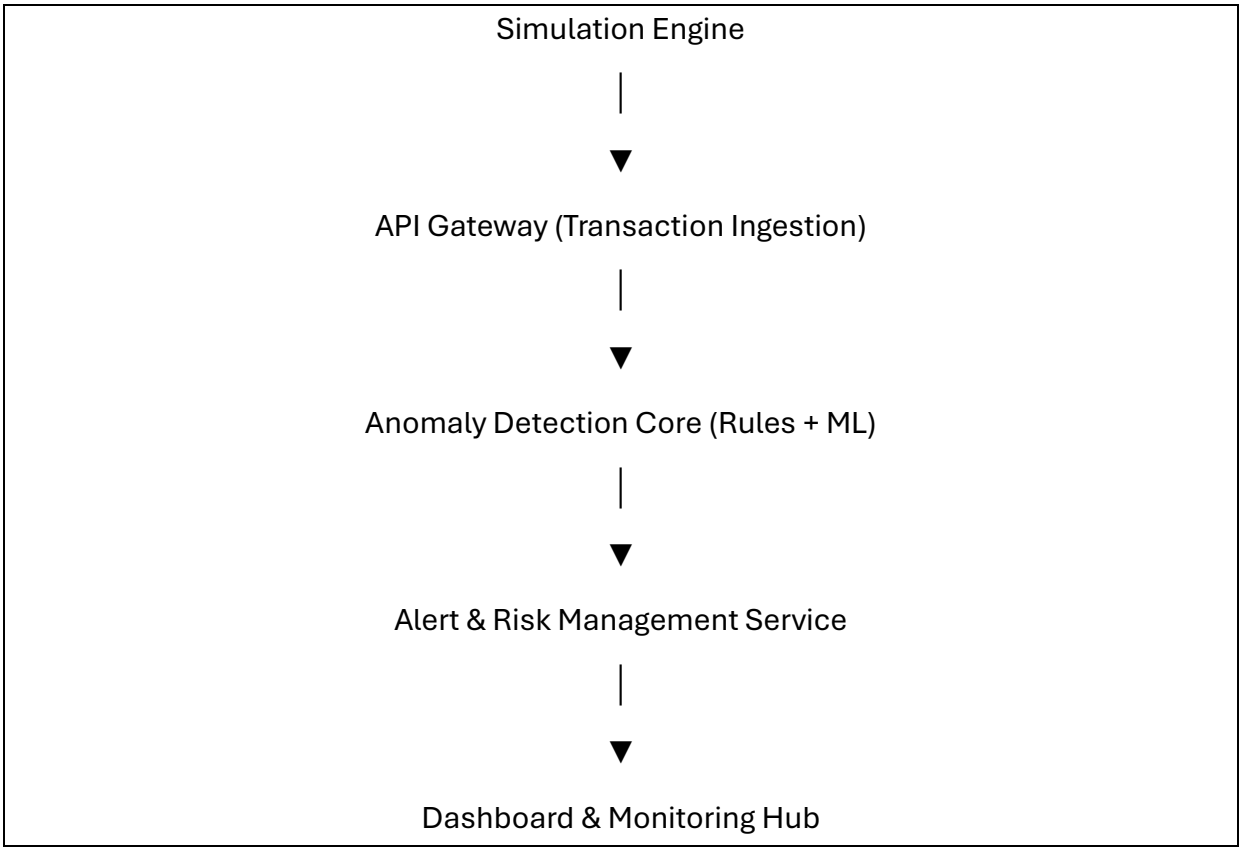
The system should:

- Accept online banking transactions (real or simulated)
- Detect suspicious activity
- Generate fraud alerts
- Provide monitoring dashboard
- Allow ML-based risk scoring

---

2 High-Level System Architecture

2.1 Logical Architecture



2.2 Data Flow Summary

1. Transaction generated (simulation or external source)
2. API Gateway receives transaction
3. Transaction stored in database
4. Detection Core evaluates risk
5. If fraud → Alert Service stores alert
6. Dashboard fetches alerts & metrics

---

3. Standard Transaction Data Model (DO NOT MODIFY)

⚠️ This structure must remain consistent across all modules.

```
{
"transactionId": "STRING",
"accountId": "STRING",
"customerId": "STRING",
"transactionType": "DEBIT/CREDIT",
"channel": "MOBILE_APP/WEB/ATM",
"amount": 0.0,
"currency": "INR/USD",
"merchantId": "STRING",
"merchantCategory": "STRING",
"merchantCountry": "STRING",
"customerCountry": "STRING",
"deviceId": "STRING",
"ipAddress": "STRING",
"timestamp": "YYYY-MM-DDTHH:MM:SS"
}
```

Do not change the following:

- Rename fields
- Remove fields
- Add required fields

---

4 Module Responsibilities & Ownership

Member	Module	Responsibilities
Member 1	Transaction API Gateway	Accept, validate & store transactions
Member 2	Simulation Engine	Generate normal & fraud transactions

Member	Module	Responsibilities
Member 5	Anomaly Detection Core	Rule engine + ML integration
Member 3	Alert & Analytics Service	Store alerts & expose monitoring APIs
Member 4	ML Plugin Service	Train model & provide fraud probability API

---

5 Module Specifications

🔹 MODULE 1 – Transaction API Gateway

Purpose

Acts as the system entry point.

Responsibilities

- Expose transaction ingestion API
- Validate transaction JSON
- Store in database
- Forward transaction to Detection Core

Database Schema

```
CREATE TABLE transactions (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  transaction_id VARCHAR(50) UNIQUE,
  account_id VARCHAR(50),
  customer_id VARCHAR(50),
  transaction_type VARCHAR(20),
  channel VARCHAR(30),
  amount DECIMAL(15,2),
  currency VARCHAR(10),
  merchant_id VARCHAR(50),
  merchant_category VARCHAR(50),
```

```
  merchant_country VARCHAR(50),
  customer_country VARCHAR(50),
  device_id VARCHAR(100),
  ip_address VARCHAR(50),
  timestamp DATETIME,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

APIs

POST /api/v1/transactions

Response:

```
{
 "status": "SUCCESS",
 "message": "Transaction stored"
}
```

Temporary Handling

If Detection Core is not ready:

- Log forwarding action
- Use mock method
- Do not block development

🔹 MODULE 2 – Transaction Simulation Engine

Purpose

Generate real-world banking activity.

Responsibilities

- Generate normal transactions
- Generate fraud scenarios
- Call API Gateway endpoint

Fraud Scenarios

1. High Amount (> ₹100000)
2. Impossible Travel (country mismatch)
3. Rapid Transactions (5 within 1 min)
4. High Risk Merchant Category (CRYPTO, GAMBLING)
5. New Device ID

Communication

Must call:

```
POST /api/v1/transactions
```

No direct DB access allowed.

Temporary Handling

If API Gateway not ready:

- Print generated JSON
- Store locally for testing

🔹 MODULE 3 – Anomaly Detection Core

Purpose

Core fraud detection logic.

Responsibilities

- Accept transaction internally
- Apply rule engine
- Call ML service
- Calculate final risk score
- Send alert if risk > threshold

Rule Engine

Rule	Condition
HIGH_AMOUNT	amount > 100000

Rule	Condition
IMPOSSIBLE_TRAVEL	merchantCountry != customerCountry
RAPID_TXN	5 in 60 seconds
HIGH_RISK_CATEGORY	CRYPTO/GAMBLING
NEW_DEVICE	unseen deviceId

Risk Calculation

```
Final Risk = (Rule Score × 0.7) + (ML Score × 0.3)

Threshold: 70
```

Temporary Handling

If ML not ready:

- Generate random probability

If Alert service not ready:

- Print alert to console

🔹 MODULE 4 – Alert & Analytics Service

Purpose

Store fraud alerts and provide monitoring data.

Database Schema

```
CREATE TABLE alerts (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  transaction_id VARCHAR(50),
  account_id VARCHAR(50),
  risk_score INT,
  fraud_probability DOUBLE,
  rules_triggered VARCHAR(255),
  status VARCHAR(30),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

```
);
```

APIs

```
GET /api/v1/alerts
GET /api/v1/alerts/high-risk
GET /api/v1/alerts/by-account/{accountId}
GET /api/v1/metrics
```

Temporary Handling

If Detection Core not ready:

- Manually test via Postman
- Insert test alert JSON

🔹 MODULE 5 – ML Plugin Service

Purpose

Provide fraud probability prediction.

Responsibilities

- Export transaction dataset
- Label fraud cases
- Train Random Forest model
- Expose prediction API

API

```
POST /ml/predict
```

Response:

```
{
  "fraudProbability": 0.82
}
```

Temporary Handling

If Detection not ready:

- Test independently via Postman

---

6 Development Rules

1. Do NOT change JSON structure.
2. Do NOT directly access another module's database.
3. All communication via REST APIs only.
4. Mock missing services.
5. Document APIs before integration.

---

7 Integration Plan

Order of Integration

1. API Gateway
2. Simulation Engine
3. Detection Core
4. Alert Service
5. ML Integration
6. Dashboard

Replace mock calls with real endpoints during integration.

---

9 Tech Stack:

- Backend: Spring Boot
- Database: MySQL
- ML Service: Python FastAPI
- Frontend: React
- Local deployment for demo